

实验6实验报告

实验内容

1. 控制器设计实验

实验内容

根据 RV32I 指令控制信号列表，写出各个控制信号对应的逻辑表达式

整体方案设计

输入输出引脚：

输入：

opcode：7位输入的操作码

func3：三位功能码，补充opcode相同的指令的功能

func7：七位功能码，补充opcode和func3均相同的指令的功能

输出：

ExtOp：宽度为 3 位，选择立即数产生器的输出类型。

RegWr：宽度为 1 位，控制是否对寄存器 rd 进行写回，为 1 时写回寄存器。

ALUASrc 宽度为 1 位，选择 ALU 输入端 A 的来源。为 0 时选择 BusA，为 1 时选择 PC。

ALUBsrc 宽度为 2 位，选择 ALU 输入端 B 的来源。为 00 时选择 BusB，为 01 时选择常数 4，为 10 时选择立即数Imm

ALUctr：宽度为 4 位，选择 ALU 执行的操作。

Branch：宽度为 3 位，说明跳转指令的类型，用于生成最终的分支控制信号。

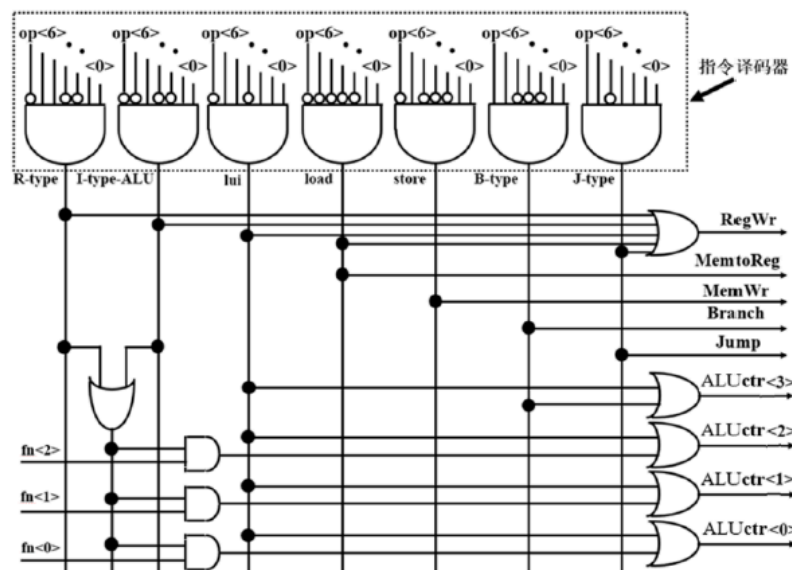
MemtoReg：宽度为 1 位，选择寄存器 rd 写回数据来源，为 0 时选择 ALU 输出，为 1 时选择数据存储器输出。

MemWr：宽度为 1 位，控制是否对数据存储器进行写入，为 1 时写回存储器。

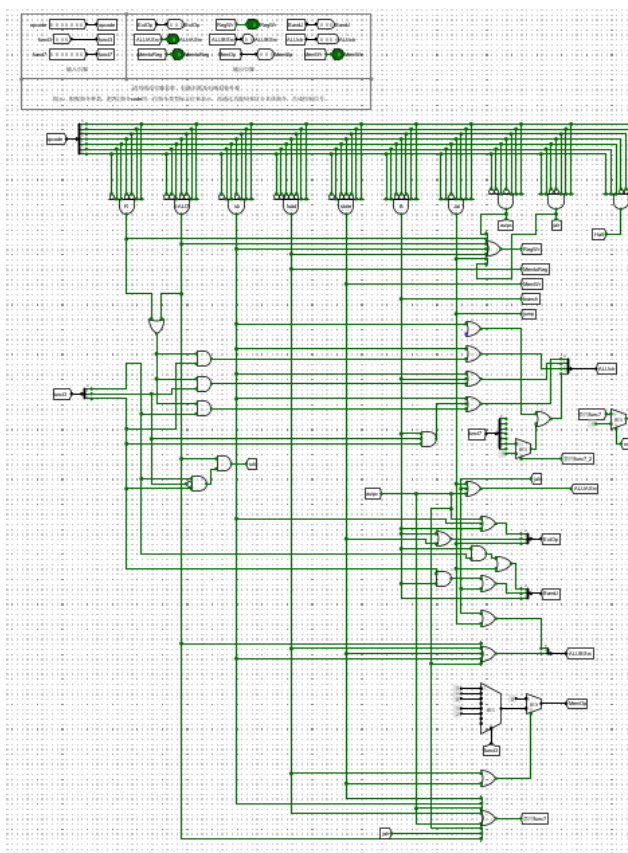
MemOp：宽度为 3 位，控制数据存储器读写格式。

原理图和电路图

原理图如下：（仅示意，实际考虑的情况比图中多）

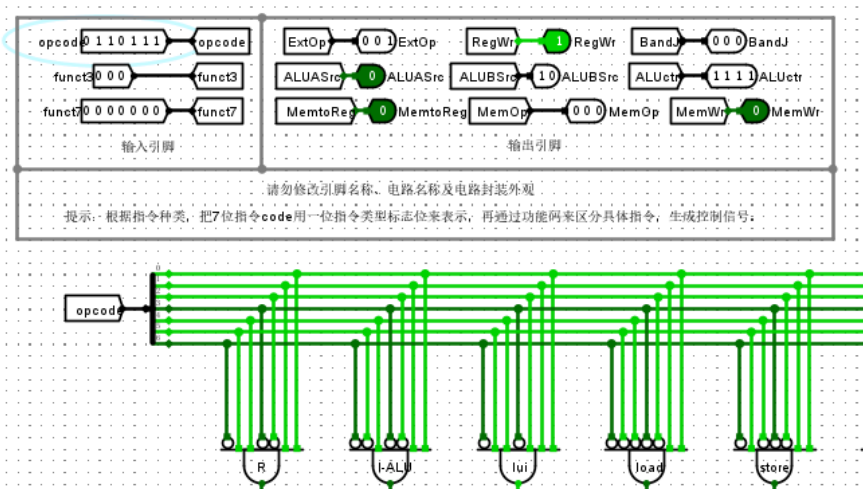


电路图如下：

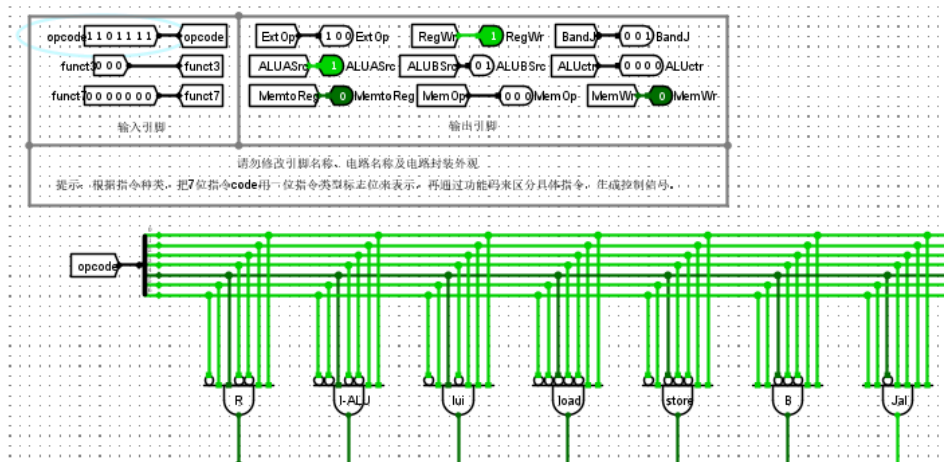


仿真测试图

opcode=0110111, 代表lui指令, 看到电路中代表lui指令的线亮起, 输出的各个控制信号符合预期要求, 正确。



opcode=1101111，代表jal指令，看到电路中代表jal指令的线亮起，输出的各个控制信号符合预期要求，正确。



错误分析

在完成实验的过程中，没有遇到任何错误。

2. 单周期CPU设计实验

实验内容

设计一个单周期CPU：根据单周期 CPU的设计，每个时钟周期完成一条指令的执行。在本次实验中，以时钟下降沿作为每个时钟周期的开始，写入操作在时钟下降沿时同步实现。读取操作则是异步实现，只要输入有效地址之后，立即输出对应的数据。

整体方案设计

输入输出引脚：

输入：

- Init addr：决定初始PC指向的值
- RESET：决定是否加载init addr，为1时，加载init addr到PC中。
- clk：时钟

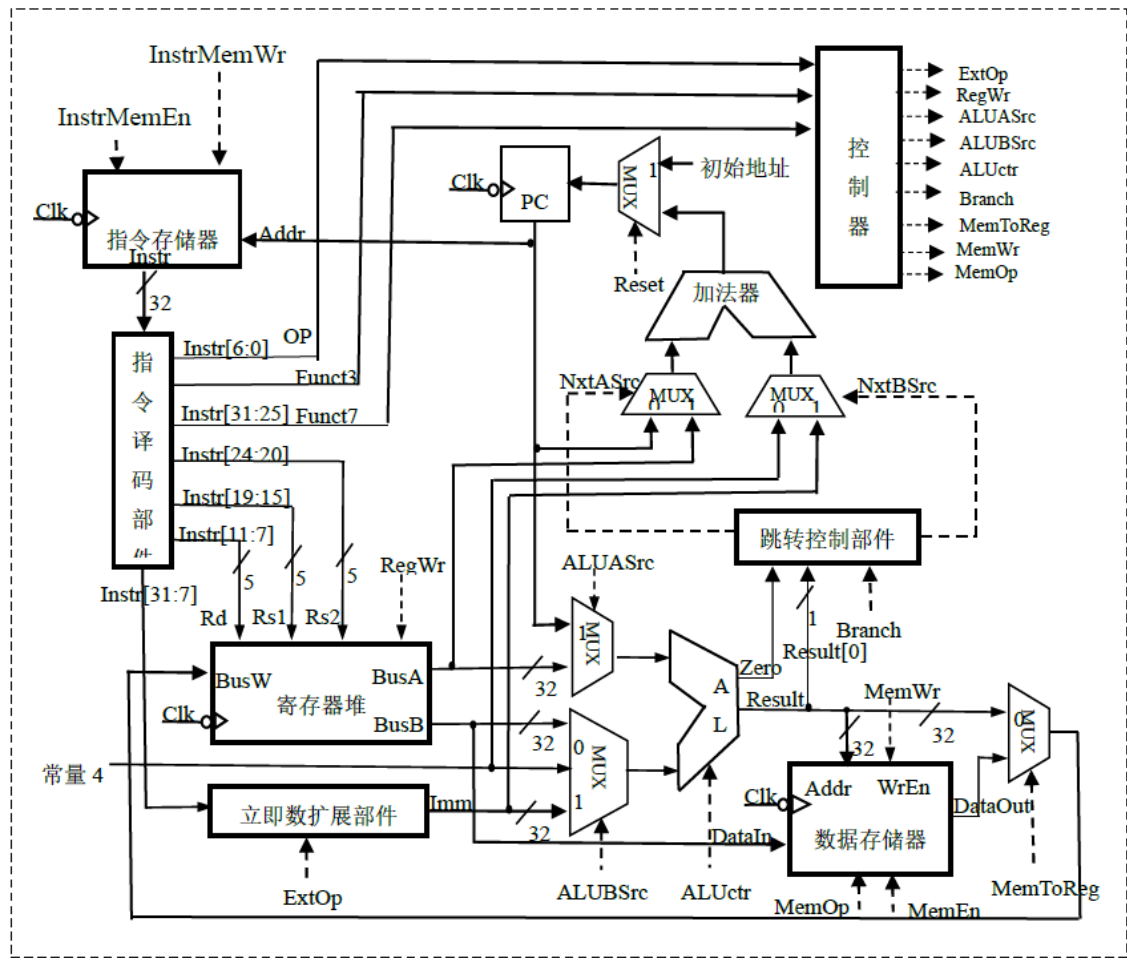
输出：

- PC：当前的程序计数器值
- IR：当前的指令值

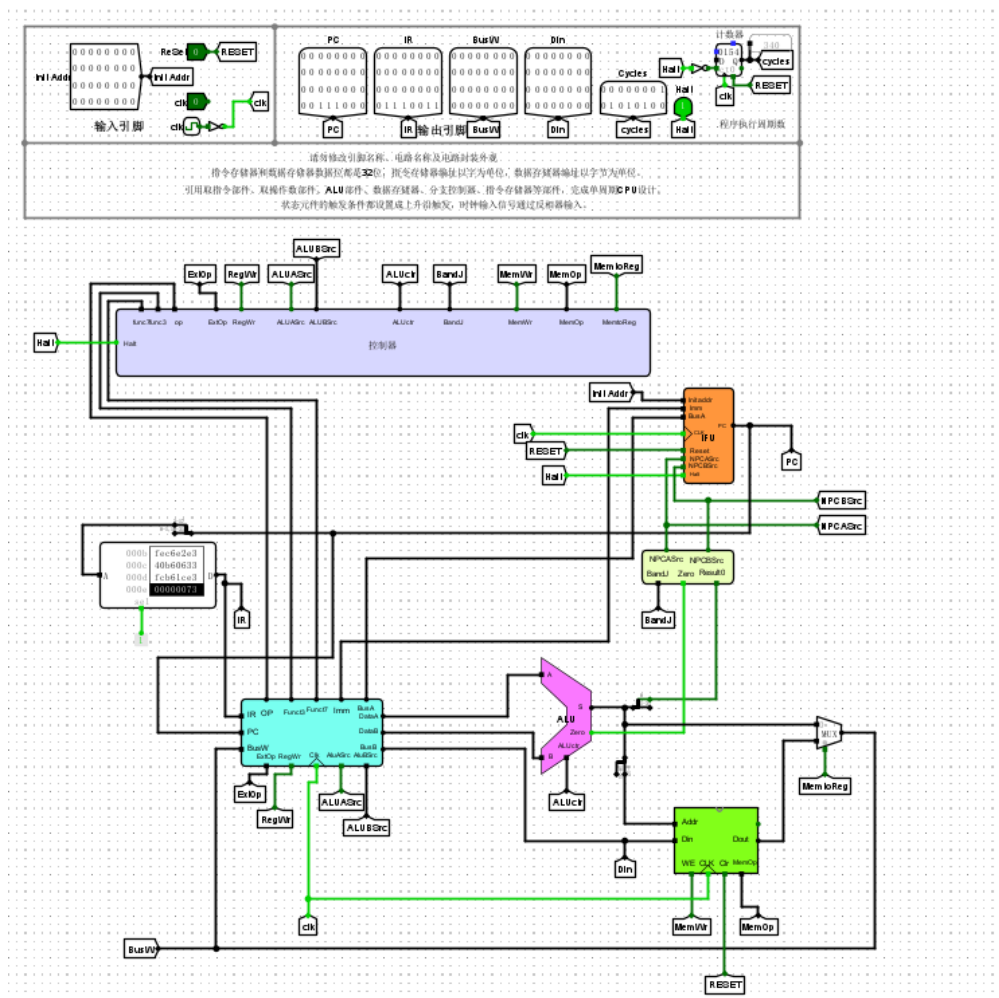
- BusW：当前在数据通路上的值，也即即将写入寄存器堆的值
- Din：即将写入数据存储器的值

原理图和电路图

实验原理图如下：

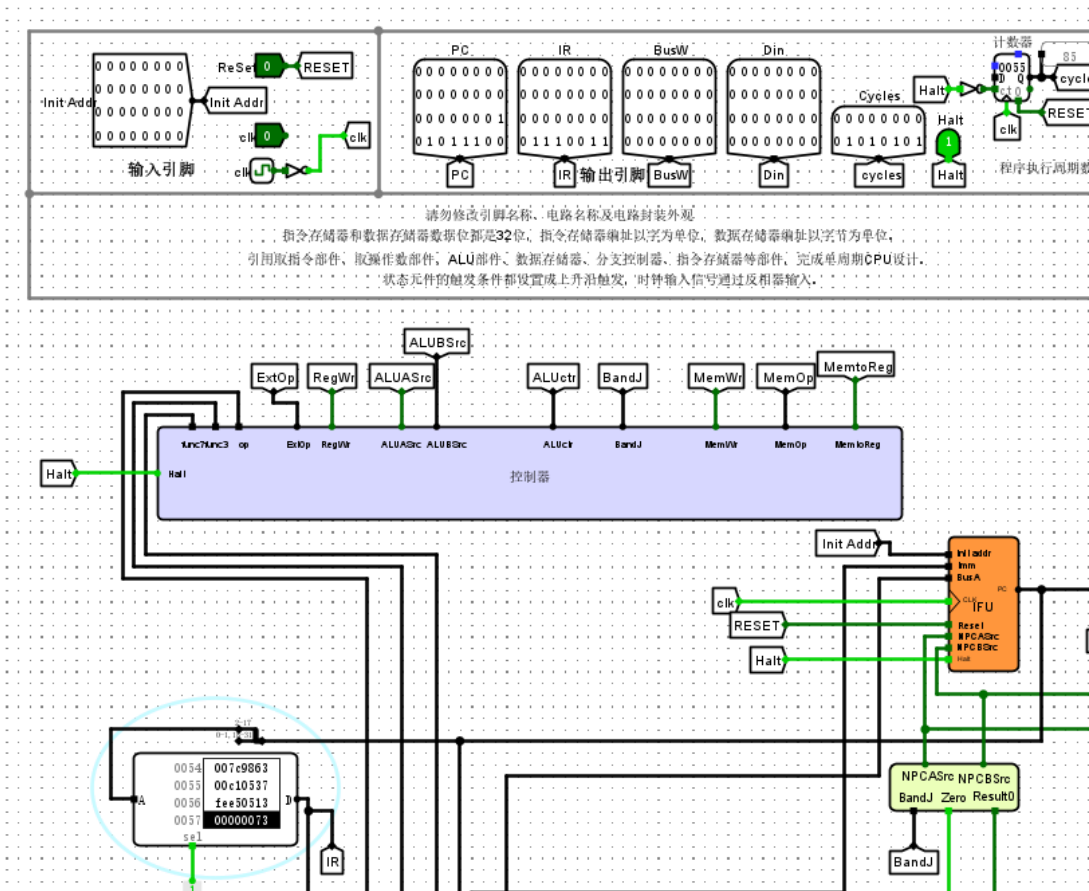


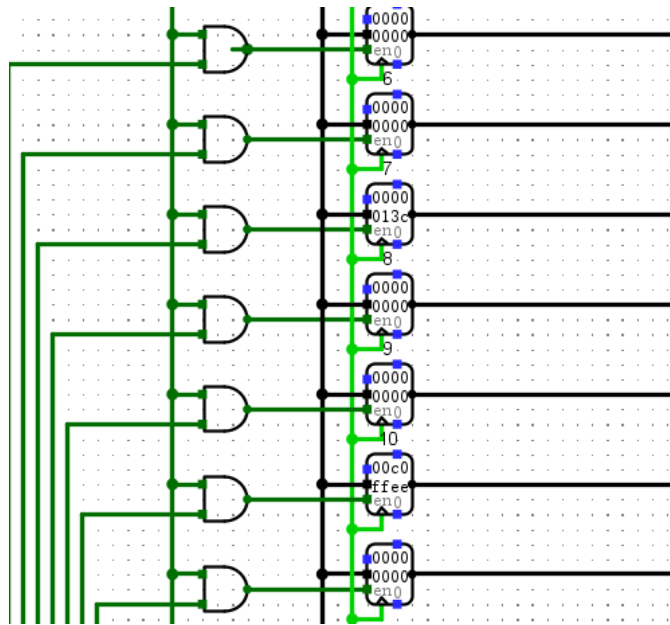
实验电路图如下：



仿真测试图

加载lab6.2.hex，运行结果如下：





10号寄存器中值为coffee，故正确。

错误分析

在完成实验的过程中，没有遇到任何错误。

3. 用累加程序验证CPU设计

实验内容

首先借助 RARS 软件得到某一段程序的机器二进制指令。然后给定一组测试数据。把二进制指令加载进指令存储器，把待测数据加载进数据存储器。采用的指令是累加程序。

原理图和电路图

与第二问完全相同，故不再重复放图片了。

仿真测试图

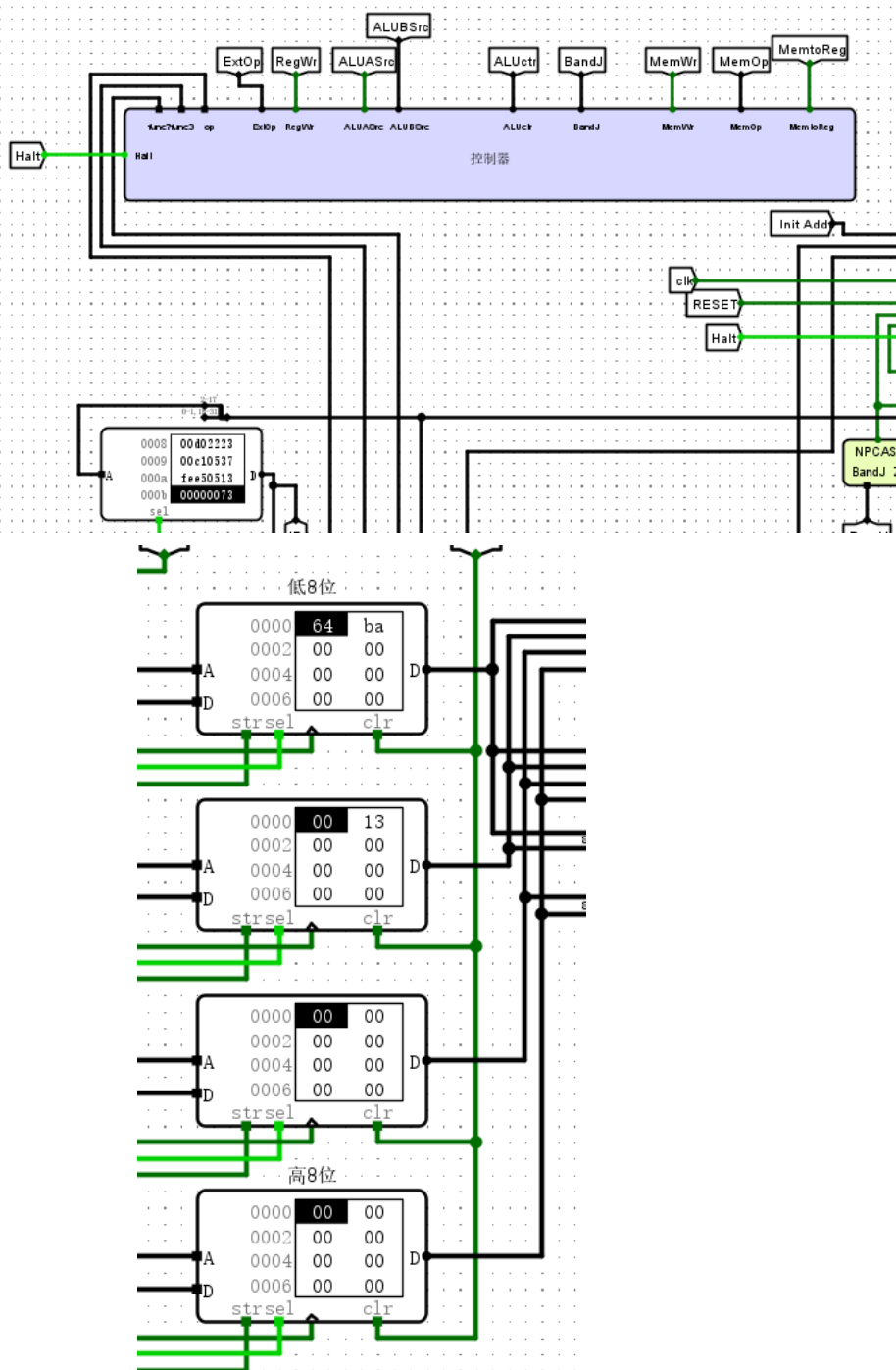
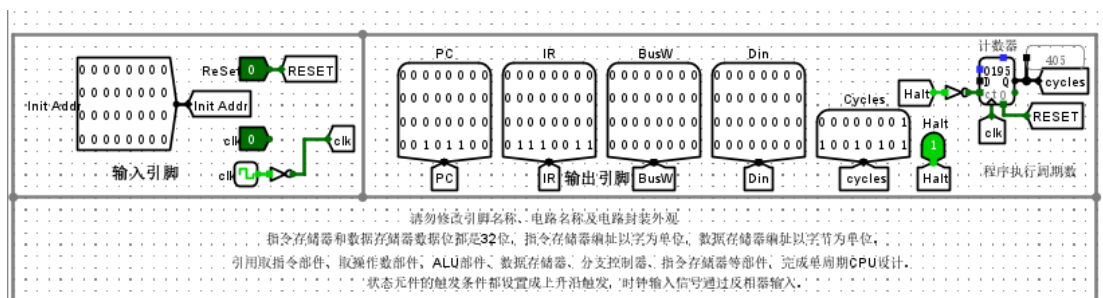
- 生成的.hex文件：

```

ASM lab6.3.asm    lab6.3.hex X
lab6.3.hex
1  v2.0 raw
2  00002583
3  02058663
4  00106613
5  00d6c6b3
6  00c686b3
7  00b60663
8  00160613
9  ff5ff06f
10 00d02223
11 00c10537
12 fee50513
13 00000073
14 deade537
15 ead50513
16 00000073
17

```

- 加载指令和数据后，在电路中执行的结果：



得到的结果为0x13ba，和预期相同。

错误分析

在完成实验的过程中，没有遇到任何错误。

4. 用冒泡排序程序进行CPU设计验证

实验内容

首先借助 RARS 软件得到某一段程序的机器二进制指令。然后给定一组测试数据。把二进制指令加载进指令存储器，把待测数据加载进数据存储器。采用的指令是冒泡排序

原理图和电路图

与第二问完全相同，故不再重复放图片了。

仿真测试图

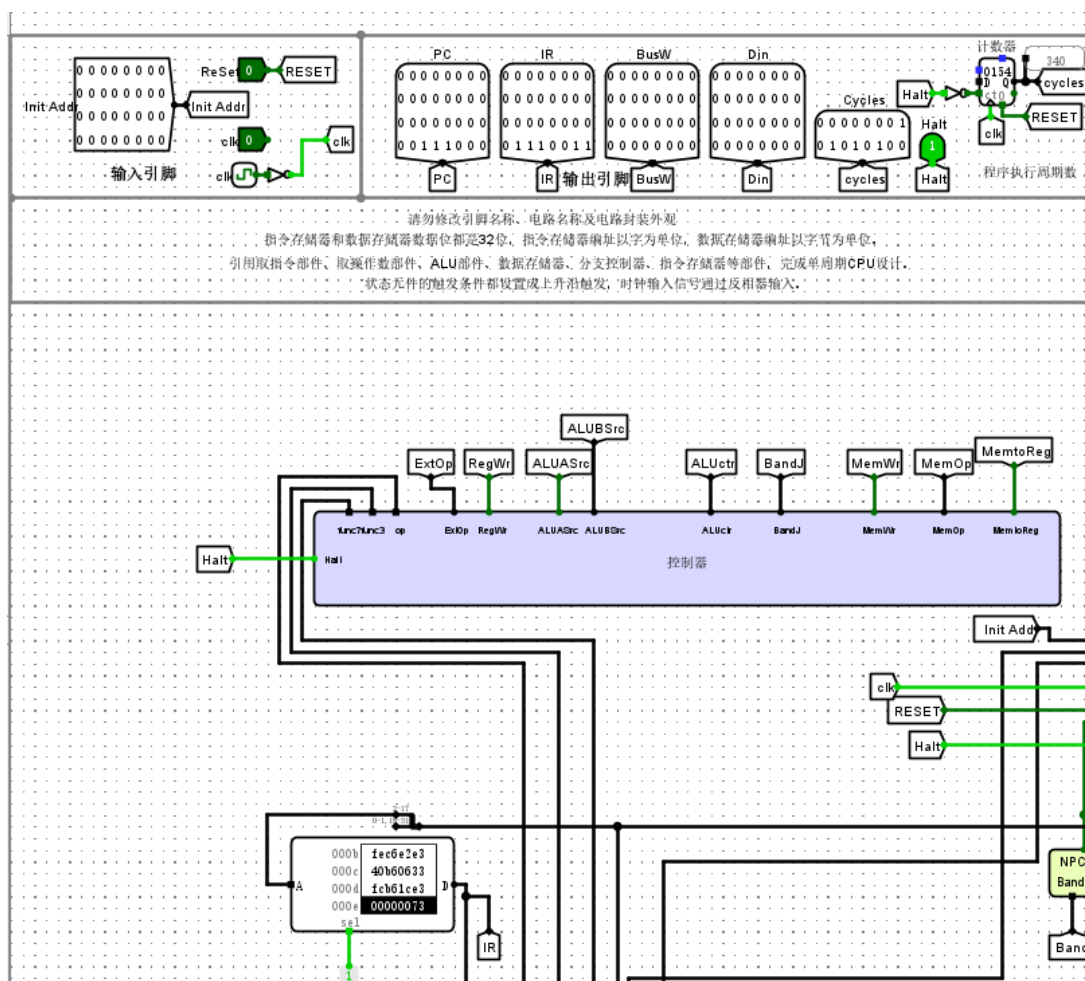
生成的.hex文件：

```
lab6.4.hex
1  v2.0 raw
2  00002283
3  00100593
4  00028633
5  00100693
6  00269713
7  00072803
8  00472883
9  0108d663
10 01172023
11 01072223
12 00168693
13 fec6e2e3
14 40b60633
15 fcb61ce3
16 00000073
17
```

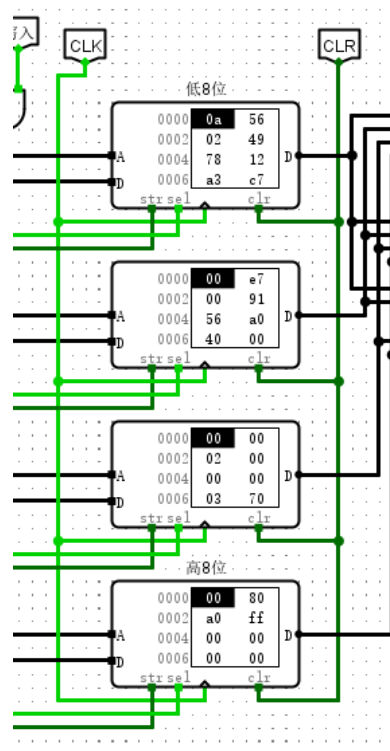
在RARS中仿真执行的结果：

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00000000	0x0000000a	0x8000e756	0xa0020002	0xff009149	0x00005678	0x0000a012	0x000340a3	0x007000c7
0x00000020	0x00800adb	0x00d00205	0x07001234	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x000000a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x000000c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x000000e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

在电路中执行.hex文件的结果：



数据存储器中的结果，和预期相同：



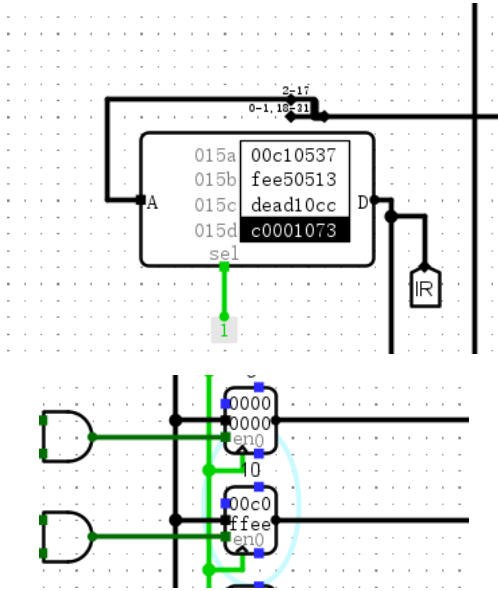
错误分析

在完成实验的过程中，没有遇到任何错误。

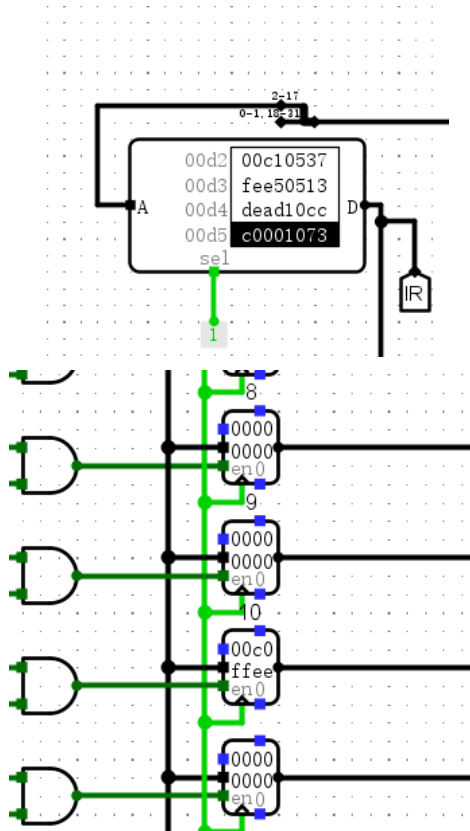
5. 官方测试集验证

由于官方测试集中的指令非常多，这里仅展示一部分指令的验证情况。

1. add指令：



2. beq指令：

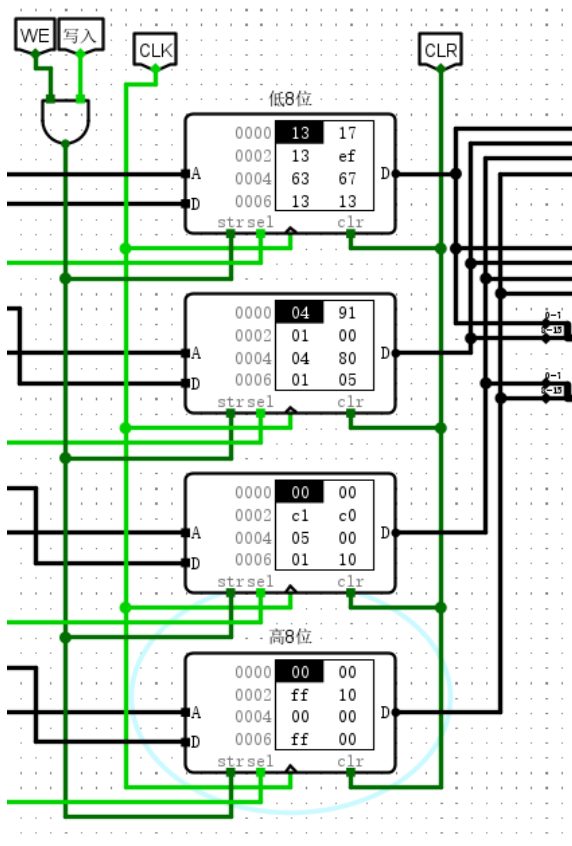


3. lb指令，与之前两条不同，load类型指令，不仅需要把.hex文件加入指令存储器中，还要在数据存储器中加载测试数据。

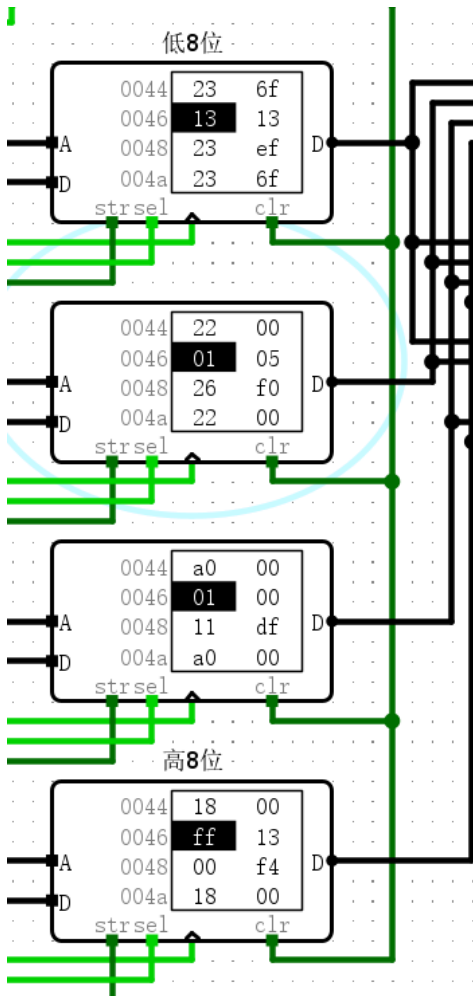
执行结果如下：

1. 测试bubble-sort-riscv32-npc.bin-logisim-inst, 需要将指令和数据分别加载进入指令存储器和数据存储器中。

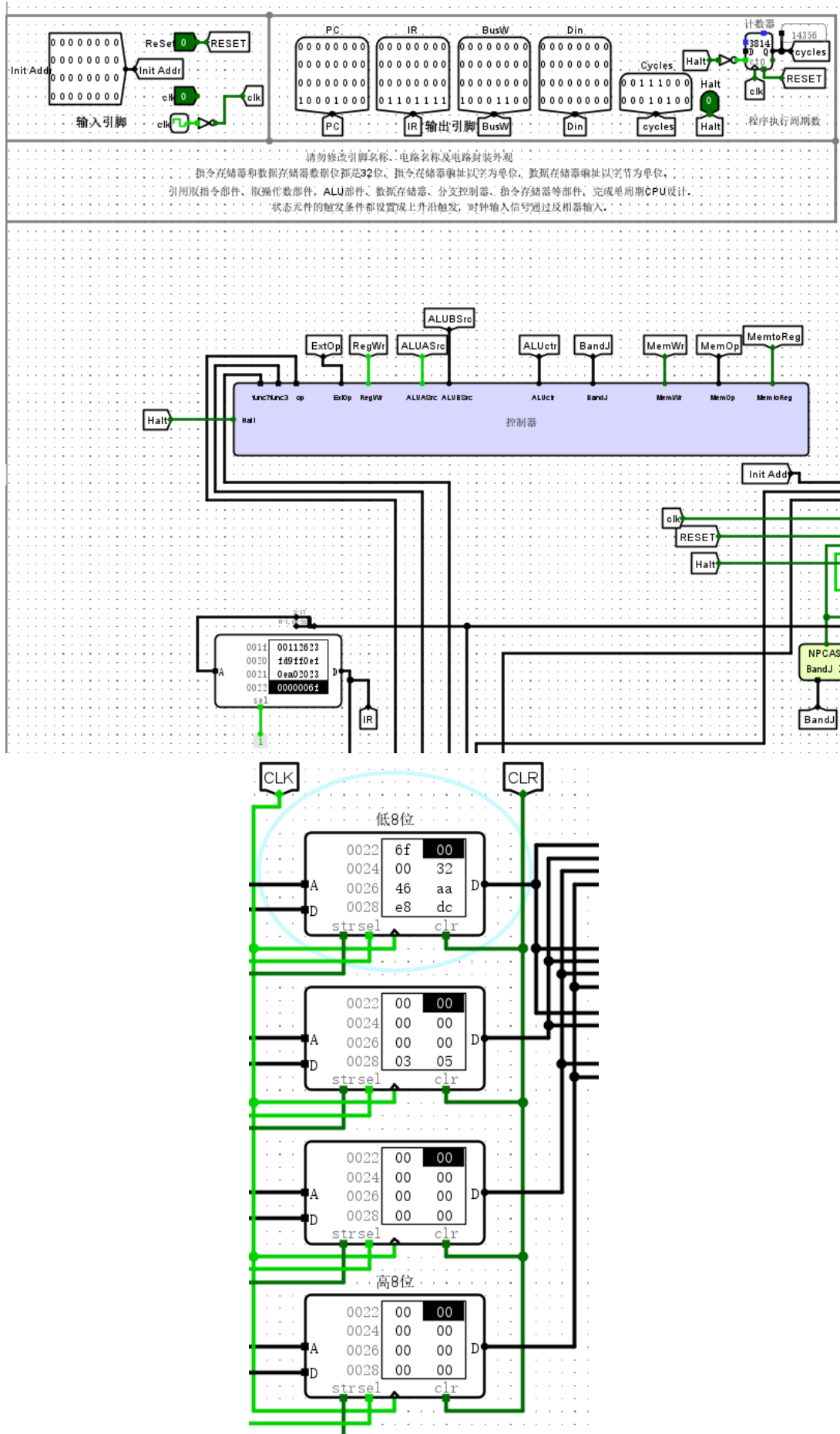
数据存储器中的加载情况如下图：



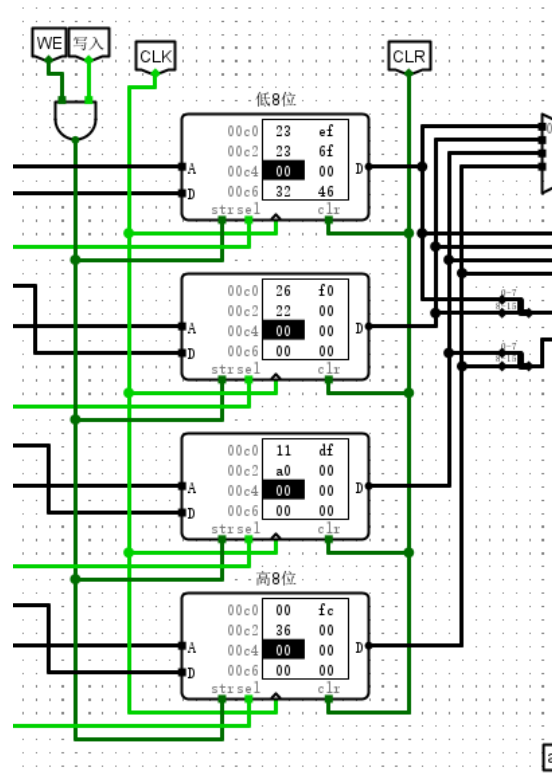
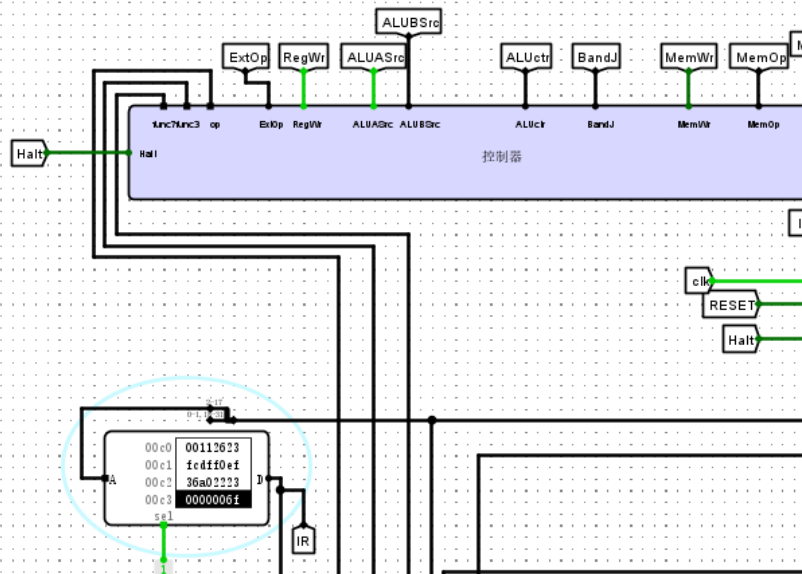
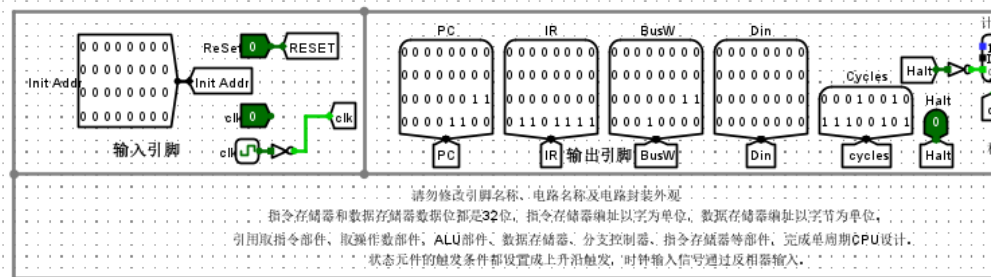
运行后的数据存储器值：



2. 测试B-sort:



3. q-sort:



错误分析

在完成实验的过程中，没有遇到任何错误。

思考题

T1 如何在单CPU上实现多任务处理

因为单周期CPU的每个指令都在一个时钟周期内完成，所以真正的并行处理是不可能的，但可以采用“任务切换”方式来模拟多任务的执行。

思路：模拟任务调度，即通过轮流执行不同程序的一部分，来模拟多任务处理。由于是单周期CPU，因此每次只能执行一条指令，但可以通过切换程序计数器（PC）的值和堆栈来进行任务切换。

如何实现任务切换：

1. 将多个程序存放在不同的内存区域。例如，程序A（计算累加）存储在内存的地址0x1000-0x1FFF区域，程序B（数据排序）存储在地址0x2000-0x2FFF区域。
2. 通过一个“时间片”机制来进行任务切换，例如每执行一定数量的指令后就切换到另一个任务。
3. 在程序执行过程中，PC会根据当前任务的指令位置不断增加。如果达到一个预设的时间片（例如，执行10条指令），就通过某种方式（例如一个定时器）触发任务切换。
4. 任务切换时，保存当前任务的状态（例如，PC值，寄存器值等）到一个特定的内存区域或堆栈中，然后将程序计数器（PC）指向下一个任务的入口地址。

T2 实现输入输出设备的数据访问

CPU与外设通信的常用方式有端口映射和内存映射两种：

- 端口映射：将外设的数据、状态、控制寄存器称为I/O端口。对端口进行编号，CPU使用in与out指令同端口间通过按编号“打电话”的方式通信
- 内存映射：将数据存储在内存中的一个特定地址（比如 0xF000），CPU通过读取这个地址来获取按键/显存/TTY信息

根据这两种思想，实现键盘输入的思路如下：

1. **键盘扫描**：设计一个键盘扫描电路，当键盘上的按键被按下时，控制器会生成一个中断请求（IRQ），并将按键的扫描码存储到一个缓冲区中。
2. **键盘数据读取**：CPU可以在需要时读取缓冲区中的按键数据。可以通过检查一个状态寄存器来判断是否有新的按键输入。

实现TTY输入：

1. **输出缓冲区**：设计一个输出缓冲区来存储字符数据。当CPU需要向屏幕显示文本时，它会将字符写入缓冲区。
2. **数据传输**：当缓冲区被填满时，TTY设备将开始将缓冲区中的字符逐个发送到显示器上。显示完成后，缓冲区为空，CPU可以继续写入新的字符。

通过内存映射I/O或端口映射I/O，CPU可以读取键盘输入并将数据发送到TTY设备输出，从而构建一个基本的计算机系统。

T3 如何在单周期CPU基础上实现多周期CPU和流水线CPU

多周期

思想：通过将每条指令分解为多个阶段，每个阶段使用一个独立的时钟周期来完成。每条指令的执行被划分为多个步骤，每个步骤在不同的时钟周期内执行。

设计思路：

- 每条指令分成多个阶段，每个阶段在一个时钟内完成
- 不同指令包含的时钟个数不同
- 阶段的划分要均衡，每个阶段只能完成一个独立、简单的功能，如：
 - 一次ALU操作
 - 一次存储器访问
 - 一次寄存器存取
- 加临时寄存器存放指令执行的中间结果
- 同一个功能部件能在不同的时钟中被重复使用
- 用有限状态机来表示指令执行流程，并以此设计控制器

流水线

思想：

- 均衡时指令吞吐率提高N倍，但不能缩短一条指令的执行时间
- 流水段数以最复杂指令所需步骤数为准（有些指令的某些阶段为空操作），每个阶段的宽度以最复杂阶段所需时间为准

具体到RV32中，以Load指令为准，分为五个阶段：取指令段IF，译码/读寄存器段ID，执行段EX，存储器段M，写回寄存器段WB。

实现：要实现专门的流水线控制器

- ID段生成所有控制信号，并随指令执行过程信息同步向后续阶段流动
- 与单周期处理器的控制器的实现方法一样，无需采用有限状态机

实验六 线下验收表

序号	测试指令	x1 寄存器输出值	x3 寄存器输出值	x10 寄存器输出值	时钟周期数
1	add	0x10	0x26	0xc0ffee	458
2	addi	0x21	0x19	0xc0ffee	235
3	and	0x11111111	0x1b	0xc0ffee	478
4	andi	0x00ff00ff	0xe	0xc0ffee	191
5	auipc	0x0	0x3	0xc0ffee	52
6	beq	0x3	0x15	0xc0ffee	284
7	bge	0x3	0x18	0xc0ffee	302
8	bgeu	0x3	0x18	0xc0ffee	327
9	blt	0x3	0x15	0xc0ffee	284
10	bltu	0x3	0x15	0xc0ffee	309
11	bne	0x3	0x15	0xc0ffee	284
12	jal	0x3	0x3	0xc0ffee	48
13	jalr	0x0	0x7	0xc0ffee	108
14	lb	0x2000	0x13	0xc0ffee	238
15	lbu	0x2000	0x13	0xc0ffee	238
16	lh	0x2000	0x13	0xc0ffee	250
17	lhu	0x2000	0x13	0xc0ffee	257
18	lui	0xffff800	0x6	0xc0ffee	58
19	lw	0x2000	0x13	0xc0ffee	260
20	or	0x11111111	0x1b	0xc0ffee	481
21	ori	0x00ff00ff	0xe	0xc0ffee	198
22	sb	1	17	0xc0ffee	423
23	sh	3001	17	0xc0ffee	476
24	sll	400	2b	0xc0ffee	486
25	slli	21	19	0xc0ffee	234
26	slt	10	26	0xc0ffee	452
27	slti	00ff00ff	19	0xc0ffee	230

28	sltiu	0x00ff00ff	0x19	0xc0ffee	230
29	sltu	0x10	0x26	0xc0ffee	452
30	sra	0x0400	0x2b	0xc0ffee	505
31	srai	0x21	0x19	0xc0ffee	249
32	srl	0x0400	0x2b	0xc0ffee	499
33	srli	0x21	0x19	0xc0ffee	243
34	sub	0x10	0x25	0xc0ffee	450
35	sw	0x12233001	0x17	0xc0ffee	483
36	xor	0x11111111	0x1b	0xc0ffee	480
37	xori	0x00ff00ff	0xe	0xc0ffee	200
C-Test C 语言测试程序执行结果					
1	b-sort.c	指令存储器加载: b-sort-riscv32-npc.bin-logisim-inst.txt, 数据存储器分别加载: b-sort-riscv32-npc.bin-logisim-data0.txt ~ b-sort-riscv32-npc.bin-logisim-data3.txt			
程序执行周期数:		1348			
程序执行结果, 数据存储器地址 M[0x90-0xDC]中的数据排列结果:					
0x0,0x32,0x46,0xaa,0x03e8,0x05dc,0x1f40,0x2af8,0x4e20,0x015f90,0x0186a0,0x01d4a0,0x02bf20,0x061a80,0x13d620,0x2dc6c0,0x5b8d80,0x09896800,0x0b532b80,0x53724e00					
2	q-sort.c	指令存储器加载: q-sort-riscv32-npc.bin-logisim-inst.txt, 数据存储器分别加载: q-sort-riscv32-npc.bin-logisim-data0.txt ~ q-sort-riscv32-npc.bin-logisim-data3.txt			
程序执行周期数:		999			
程序执行结果, 数据存储器地址 M[0x314-0x360]中的数据排列结果:					
0x0,0x32,0x46,0xaa,0x03e8,0x05dc,0x1f40,0x2af8,0x4e20,0x015f90,0x0186a0,0x01d4a0,0x02bf20,0x061a80,0x13d620,0x2dc6c0,0x5b8d80,0x09896800,0x0b532b80,0x53724e00					